EXPERIMENT- 3

Introduction of various abstraction levels and simulation of logic circuits

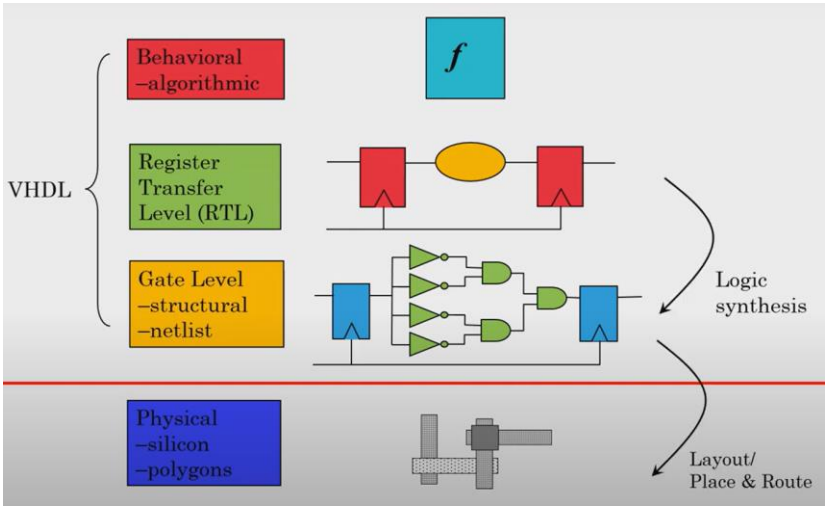
Objective:

To understand the concepts related to different abstract levels and modeling styles for logic circuits and write Verilog Programs using the same.

Theory:

Levels of Abstraction in electronic design and system modeling represent different ways of describing a system based on the amount of detail included. The abstraction level is shown below,

Level Name	Behavioral Representation	Structural Representation
System	Algorithms Truth-Tables	Processors Memories
R.T.L	Register transfers	Registers ALUs
Logic	Boolean Equations	Gates
Transistor	Transfer Function Timing diagrams	Transistors (Analog domain)



Modules are the basic building blocks for digital logic circuit modeling. The module is the principal design entity in Verilog.

Module Declaration: The first line of a module declaration specifies the *module name* and *port list* (arguments). The next few lines specify the *i/o type* (input, output or inout) and *width* of each port.

Syntax:

```
module module_name (port_list);
input [msb:lsb] input_port_list;
output [msb:lsb] output_port_list;
```

```
inout [msb:lsb] inout_port_list;
... statements...
endmodule
```

Dataflow modeling: The data-flow model uses signal assignment statements that are **concurrent** (The order of assign statements does not matter). Dataflow modeling uses *continuous assignment statements* with keyword *assign.*

assign Y = Boolean Expression using variables and operators.

A dataflow description is based on function rather than structure and hence uses a number of bit-wise operators.

Bitwise Verilog Operator	Symbol
NOT	~
AND	&
OR	|
XOR	^
XNOR	^~ or ~^

A dataflow description is based on function rather than structure and hence uses a number of bit-wise operators.

1. **Write a dataflow Verilog code for following digital building blocks and verify the design by simulation: 32:1 mux**

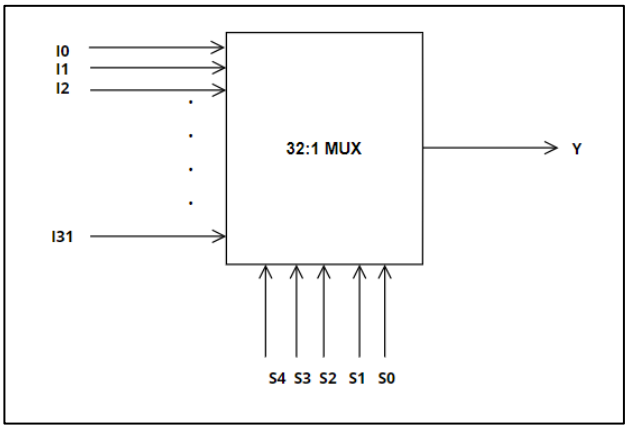
<u>32:1 mux</u>:

A 32:1 multiplexer (MUX) is a digital circuit that selects one of 32 input signals and forwards the selected input to a single output line based on the value of a set of control signals.

Components of a 32:1 MUX:

- Inputs (I0 to I31): These are the 32 data inputs, each of which can carry a binary signal (0 or 1). The MUX selects one of these inputs to send to the output.
- Control Signals (S0 to S4): These are 5 selection lines or control signals, which determine which of the 32 inputs is connected to the output. Since there are 32 inputs, you need 5 control signals.
- Output (Y): This is the single output line that carries the value of the selected input.

Circuit:



Source Code :

32mux.v
~/Abhiram_240942004/32_1MUX

```
module mux32(i,s,y);
input [31:0]i;
input [4:0]s;
output y;
assign y=i[s];
endmodule
```

Testbench:

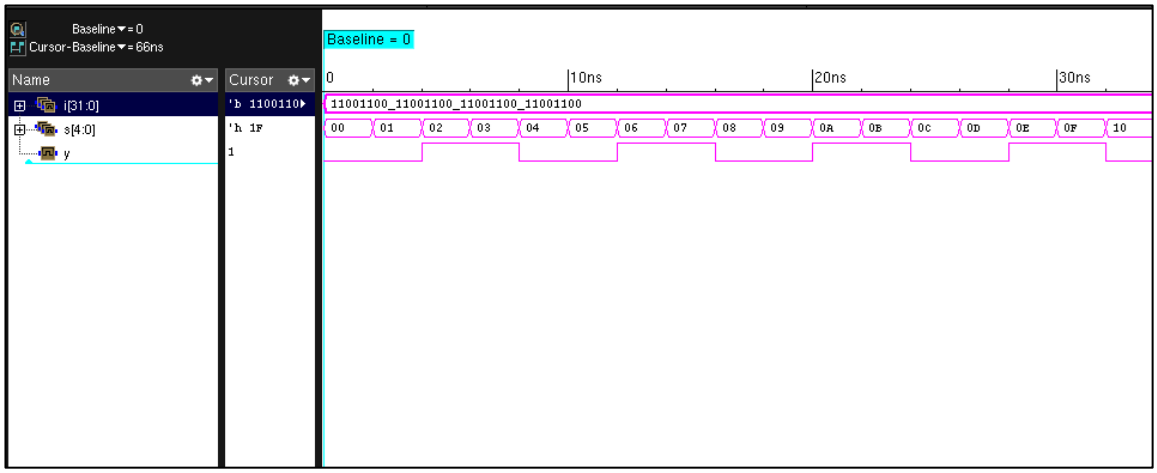
32mux_tb.v
~/Abhiram_240942004/32_1MUX

```
module mux32_tb();
reg [31:0]i;
reg [4:0]s;
wire y;

mux32 uut(.i(i),.s(s),.y(y));

initial begin
i = 32'h000000;
s = 5'b0;
i = 32'hCCCCCCCC; s = 5'd0;
#2 i = 32'hCCCCCCCC; s = 5'd1;
#2 i = 32'hCCCCCCCC; s = 5'd2;
#2 i = 32'hCCCCCCCC; s = 5'd3;
#2 i = 32'hCCCCCCCC; s = 5'd4;
#2 i = 32'hCCCCCCCC; s = 5'd5;
#2 i = 32'hCCCCCCCC; s = 5'd6;
#2 i = 32'hCCCCCCCC; s = 5'd7;
#2 i = 32'hCCCCCCCC; s = 5'd8;
#2 i = 32'hCCCCCCCC; s = 5'd9;
#2 i = 32'hCCCCCCCC; s = 5'd10;
#2 i = 32'hCCCCCCCC; s = 5'd11;
#2 i = 32'hCCCCCCCC; s = 5'd12;
#2 i = 32'hCCCCCCCC; s = 5'd13;
#2 i = 32'hCCCCCCCC; s = 5'd14;
#2 i = 32'hCCCCCCCC; s = 5'd15;
#2 i = 32'hCCCCCCCC; s = 5'd16;
#2 i = 32'hCCCCCCCC; s = 5'd17;
#2 i = 32'hCCCCCCCC; s = 5'd18;
#2 i = 32'hCCCCCCCC; s = 5'd19;
#2 i = 32'hCCCCCCCC; s = 5'd20;
#2 i = 32'hCCCCCCCC; s = 5'd21;
```

32mux_tb.v
~/Abhiram_240942004/32_1MUX

```
#2 i = 32'hCCCCCCCC; s = 5'd11;
#2 i = 32'hCCCCCCCC; s = 5'd12;
#2 i = 32'hCCCCCCCC; s = 5'd13;
#2 i = 32'hCCCCCCCC; s = 5'd14;
#2 i = 32'hCCCCCCCC; s = 5'd15;
#2 i = 32'hCCCCCCCC; s = 5'd16;
#2 i = 32'hCCCCCCCC; s = 5'd17;
#2 i = 32'hCCCCCCCC; s = 5'd18;
#2 i = 32'hCCCCCCCC; s = 5'd19;
#2 i = 32'hCCCCCCCC; s = 5'd20;
#2 i = 32'hCCCCCCCC; s = 5'd21;
#2 i = 32'hCCCCCCCC; s = 5'd22;
#2 i = 32'hCCCCCCCC; s = 5'd23;
#2 i = 32'hCCCCCCCC; s = 5'd24;
#2 i = 32'hCCCCCCCC; s = 5'd25;
#2 i = 32'hCCCCCCCC; s = 5'd26;
#2 i = 32'hCCCCCCCC; s = 5'd27;
#2 i = 32'hCCCCCCCC; s = 5'd28;
#2 i = 32'hCCCCCCCC; s = 5'd29;
#2 i = 32'hCCCCCCCC; s = 5'd30;
#2 i = 32'hCCCCCCCC; s = 5'd31;

end

initial begin
$monitor($time,"i=%h,s=%d,y=%b",i,s,y);
#66 $finish;
end
endmodule
```

Waveform:



2. Write a dataflow Verilog code for 4-bit binary to gray code converter and verify the design by simulation.

A 4-bit Binary to Gray code converter is a digital circuit that converts a 4-bit Binary code input into its equivalent 4-bit gray code output.

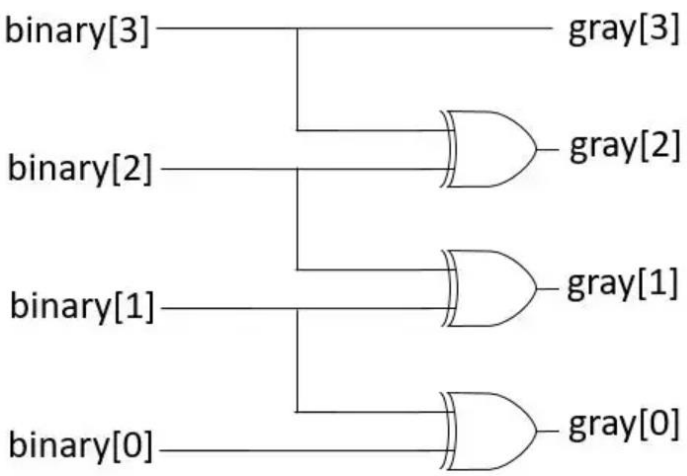
Components of a 4-bit Binary to Gray Converter:

1. The most significant bit (MSB) of the Gray code is the same as the MSB of the binary number.
2. Each subsequent bit of the Gray code is obtained by XORing the current binary bit with the previous binary bit.

Conversion Formula:

- gray[3]=binary[3]
- gray[2]= binary [3]⊕ binary [2]
- gray[1]= binary [2]⊕ binary [1]
- gray[0]= binary [1]⊕ binary [0]

Circuit:



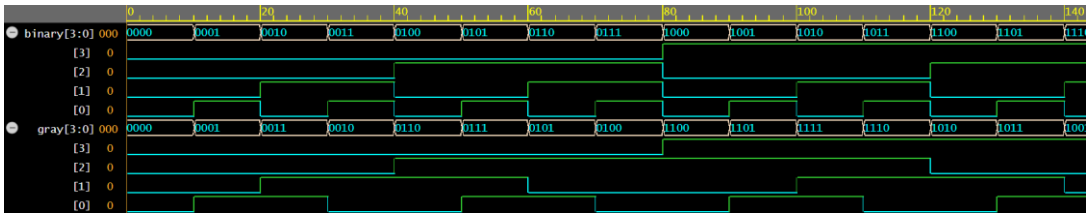
Source Code :

```
module binary_to_gray ( input  [3:0] binary,  output [3:0] gray );

   assign gray[3] = binary[3];                  // MSB remains the same
   assign gray[2] = binary[3] ^ binary[2];   // XOR of bit 3 and bit 2
   assign gray[1] = binary[2] ^ binary[1];   // XOR of bit 2 and bit 1
   assign gray[0] = binary[1] ^ binary[0];   // XOR of bit 1 and bit 0

endmodule
```

Testbench:

```
module tb_binary_to_gray;
   reg  [3:0] binary;
   wire [3:0] gray;

   // Instantiate the binary_to_gray module
   binary_to_gray uut ( .binary(binary), .gray(gray));

   initial begin
      $monitor("Binary = %b, Gray = %b", binary, gray);

      // Test cases
      binary = 4'b0000; #10;
      binary = 4'b0001; #10;
      binary = 4'b0010; #10;
      binary = 4'b0011; #10;
      binary = 4'b0100; #10;
      binary = 4'b0101; #10;
      binary = 4'b0110; #10;
      binary = 4'b0111; #10;
      binary = 4'b1000; #10;
      binary = 4'b1001; #10;
      binary = 4'b1010; #10;
      binary = 4'b1011; #10;
      binary = 4'b1100; #10;
      binary = 4'b1101; #10;
      binary = 4'b1110; #10;
      binary = 4'b1111; #10;

      $finish;
   end

endmodule
```

Waveform:



Exercise Problems

1. Write a dataflow Verilog code for following digital building blocks and verify the design by simulation: 5:32 Decoder
2. Write a dataflow Verilog code for 4-bit binary multiplier and verify the design by simulation.
3. Write a dataflow Verilog code for a BCD-to-seven-segment decoder and verify the design by simulation.
4. Write a dataflow Verilog code for binary to Excess-3 code and verify the design by simulation.
5. Write a dataflow Verilog code for 8:1 MUX with enable input and verify the design by simulation.